You are given an integer **N**, an array **A** of length **N**, and an array **B** of length **N**.

You have **N** blocks of **1 bits** with sizes **A[1...N]**, and **N** blocks of **0 bits** with sizes **B[1...N]**.You must arrange these blocks so that no two adjacent blocks have the same bit value.

Among all possible valid arrangements, find the **maximum possible number** obtained by interpreting the resulting binary string as a **decimal integer**. Print the result modulo **10^9+7.**

Find the **maximum possible number** obtained by interpreting the optimally arranged binary string as a decimal integer. Print the result modulo **10^9+7.**

Notes:

- All blocks in A and B must be used exactly once.
- Blocks must strictly alternate between 0 and 1.
- The arrangement may start with either block type.
- The objective is to maximize the decimal value of the final binary string.

Input Format

The first line contains an integer, N, denoting the number of elements in A and B.

Each line i of the N subsequent lines (where $0 \le i < N$) contains an integer describing A[i].

Each line i of the N subsequent lines (where $0 \le i < N$) contains an integer describing B[i].

vedantsaxena6987

B[i].

Constraints

1 <= **N** <= 10^5

1 <= **A[i]** <= 10^6

1 <= **B[i]** <= 10^6

Sample Test Cases

Case 1

Input:

```
1
2
1
```

Output:

```
6
```

Explanation:

```
N = 1
A = 2
B = 1
The best arrangement is A[0] then B[0] , giving the binary string 110, which is
equivalent to 6 in decimal.
```

Case 2

Input:

```
2
1
1
1
```

vedantsaxena6987

Vedant Saxena

Coding Hands-on > 2: Medium : Recamán Queries

You are given an array A of N integers and **Q** queries. Each query is either:

- **Update(i, x): Set A[i] = x.**
- **Query(L, R):** Find **maximum value M** in **[L,R]**. Compute the Recaman distance: starting from **M,** how many steps until the recaman sequence revisits a value or exceeds **10^6**? (Recaman: **R(0)=0, R(n)=R(n-1)-n** if positive and unvisited, else **R(n-1)+n)**. Return steps **×** multiplier, where multiplier = **17** if **M** is a fortunate number (smallest **m>1** where **p#+m** is prime, for some primorial **p#**), else **1.**

Find the **sum of all** query results.

Notes:

- If neither a repeat nor a value exceeding **10^6** occurs within **5000** steps, terminate at **5000.**
- Only Fortunate numbers **≤ 10000** (predefined list) are considered.
- **Recamán Sequence (in simple words):** Start from **0**. At step **n**: Try to subtract **n** from the previous number.If the result is **positive** and has **never appeared before**, accept it. Otherwise, **add n** instead. You keep generating numbers this way.
- **Recamán Distance:** For a number **M**: Start the Recamán process from **M**.
- Count how many steps it takes until: a number repeats, **or** the value becomes greater than **1,000,000**, **or** you reach **5000 steps** (stop even if nothing happened). That step count is called the **Recamán distance**.
- **Fortunate Number:** A Fortunate number is a small special number **m > 1** such that: When you add it to a product of the first few prime numbers, the result is **prime**. Only Fortunate numbers **up to 10,000** (from a fixed list) are considered.

vedantsaxena6987

You are given an array **A** of **N** integers and **Q** queries. Each query is either:

- **Update(i, x): Set A[i] = x.**
- **Query(L, R):** Find **maximum value M** in **[L,R]**. Compute the Recaman distance: starting from **M,** how many steps until the recaman sequence revisits a value or exceeds **10^6**? (Recaman: **R(0)=0, R(n)=R(n-1)-n** if positive and unvisited, else **R(n-1)+n)**. Return steps **×** multiplier, where multiplier = **17** if **M** is a fortunate number (smallest **m>1** where **p#+m** is prime, for some primorial **p#**), else **1.**

Find the **sum of all** query results.

Notes:

- If neither a repeat nor a value exceeding **10^6** occurs within **5000** steps, terminate at **5000.**
- Only Fortunate numbers **≤ 10000** (predefined list) are considered.
- **Recamán Sequence (in simple words):** Start from **0**. At step **n**: Try to subtract **n** from the previous number.If the result is **positive** and has **never appeared before**, accept it. Otherwise, **add n** instead. You keep generating numbers this way.
- **Recamán Distance:** For a number **M**: Start the Recamán process from **M**.
- Count how many steps it takes until: a number repeats, **or** the value becomes greater than **1,000,000, or** you reach **5000 steps** (stop even if nothing happened). That step count is called the **Recamán distance**.
- **Fortunate Number:** A Fortunate number is a small special number **m > 1** such that: When you add it to a product of the first few prime numbers, the result is **prime**. Only Fortunate numbers **up to 10,000** (from a fixed list) are considered.

vedantsaxena6987

Input Format

The first line contains an integer, N, denoting the size of the array A.

Each line i of the N subsequent lines (where 0 ≤ i < N) contains an integer describing A[i].

The next line contains an integer, Q, denoting the number of queries.

Each line i of the Q subsequent lines (where 0 ≤ i < Q) contain 3 space separated integers each describing the row Queries[i].

The row Queries[i] contains 3 space separated integers denoting queries in the from [0, i, x] for update and [1, L, R] for range.

Constraints

1 <= **N** <= 10^5

1 <= **A[i]** <= 10^4

1 <= **Q** <= 50000

1 <= **Queries[i][j]** <= 10^5

Sample Test Cases

Case 1

Input:

```
10
7373
5968
```

vedantsaxena6987

```
10
7373
5968
3638
47
6985
8219
59
8319
9100
9750
6
0 6 8951
1 1 7
1 2 5
1 6 8
0 4 2190
1 2 6
```

Output:

```
1233
```

Explanation:

```
Initial Array: [7373, 5968, 3638, 47, 6985, 8219, 59, 831 8951 → Contribution:
0
Range : max = 8951
Recamán steps: 243
Fortunate? No → multiplier = 1
Score: 243 × 1 = 243
Range : max = 8219
Recamán steps: 463
Fortunate? No → multiplier = 1
Score: 463 × 1 = 463
Range : max = 9100
Recamán steps: 284
Fortunate? No → multiplier = 1
Score: 284 × 1 = 284
Update: A = 2190 → Contribution: 0
Range : max = 8951
Recamán steps: 243
Fortunate? No → multiplier = 1
Score: 243 × 1 = 243
Total: 243 + 463 + 284 + 243 = 1233
```

vedantsaxena6987

Case 2

Input:

```
10
1308
398
8265
5321
9564
4620
9238
3740
1550
8924
10
1 0 0
0 1 3988
1 6 8
1 8 9
1 5 9
1 0 7
1 7 8
1 3 4
1 4 6
0 7 8327
```

Output:

```
2472
```

Explanation:

```
Initial Array: [1308, 398, 8265, 5321, 9564, 4620, 923max = 1308 → Score: 158 ×
1 = 158
2. Update: A = 3988 → 0
3. Range : max = 9238 → Score: 293 × 1 = 293
4. Range : max = 8924 → Score: 171 × 1 = 171
5. Range : max = 9238 → Score: 293 × 1 = 293
6. Range : max = 9564 → Score: 435 × 1 = 435
7. Range : max = 3740 → Score: 252 × 1 = 252
8. Range : max = 9564 → Score: 435 × 1 = 435
9. Range : max = 9564 → Score: 435 × 1 = 435
10. Update: A = 8327 → 0
Total: 158 + 293 + 171 + 293 + 435 + 252 + 435 + 435 = 2472
```

vedantsaxena6987

You are given a tree with **N** nodes, each node **v** has value **W[v]** and type **T[v]** ∈ **{0..K-1}** where **K ≤ 10**. Select a subset **S** of nodes **maximizing total value**, subject to: Independence: **No two** selected nodes share an edge.

- **Kolakoski bonus:** Let **c = |S|** (count of selected). If **c** is a position where Kolakoski sequence has value **1** (positions **1,3,5,7,10,12,14,16,19...**), add bonus = **30 × (c mod 10 + 1)**. The Kolakoski sequence **K = 1,2,2,1,1,2,1,2,2,1,2,2,1,1,2,1,1,2,2,1...** is self-describing: **K(n)** gives the length of the **n-th** run.
- **Thue-Morse penalty:** If total value **V** has Thue-Morse index **t** where **T(t)=1** (**T(n) = popcount(n)** mod **2**), subtract **25.**
- **Ore constraint:** Count nodes where **W[v]** is an Ore number (harmonic mean of divisors is integer: **1,6,28,140,270,496,672...**). If **count > K**, multiply score by **0.9**.
- **Total value** = (TotalSelectedValue + KolakoskiBonus − ThueMorsePenalty) × OreMultiplier

Find the **maximum achievable value.**

Notes:

- **Kolakoski Sequence**The **Kolakoski sequence** is a special number sequence made only of **1s and 2s**, where the sequence describes itself. **Start of the sequence:** 1, 2, 2, 1, 1, 2, 1, 2, 2, 1, 2, 2, 1, 1, 2, ...
- **Thue–Morse Sequence:** The **Thue–Morse sequence** is based on **binary digits**. Rule For a number **n**: Write **n in binary** Count how many 1s it has If the count is **even**, `T(n) = 0`If the count is **odd**, `T(n) = 1`

Input Format

vedantsaxena6987

The first line contains an integer, N, denoting the number of nodes.

Each line i of the N subsequent lines (where 0 ≤ i < N) contains an integer describing W[i].

Each line i of the N subsequent lines (where 0 ≤ i < N) contains an integer describing T[i].

The next line contains an integer, M, denoting the number of edges.

Each line i of the M subsequent lines (where 0 ≤ i < M) contains 2 space separated integers each describing the row Edges[i].

The row Edges[i] contains 2 space separated integers denoting an edge between node u to node v.

The next line contains an integer, K, denoting the number of distinct types.

Constraints

1 <= **N** <= 10^3

1 <= **W[i]** <= 2000

0 <= **T[i]** <= K

N-1 <= **M** <= N-1

1 <= **Edges[i][j]** <= N

1 <= **K** <= 10

```
9
476
1621
140
518
1519
1420
1827
6
767
0
0
0
0
0
0
0
0
0
8
0 1
1 2
0 3
0 4
2 5
2 6
5 7
3 8
1
```

Output:

```
7309
```

Explanation:

```
INPUT: N=9, W=, K=1
Ore numbers in W: 140 (node 2), 6 (node 7)
Best Independent Set: Nodes {1,4,5,6,8}
Values:
Count: 5, Total: 7154, Ore count: 0
Score Calculation:
Base: 7154
Kolakoski bonus: position 5 has value 1 → +30×(5%10+1) = +180
Thue-Morse penalty: T(7154) = 1 → -25
Ore multiplier: 0 ≤ 1, no multiplier
Final: 7154 + 180 - 25 = 7309
```

vedantsaxena6987